On Optimization Modulo Theories, MaxSMT and Sorting Networks

Roberto Sebastiani and Patrick Trentin(✉)

DISI, University of Trento, Trento, Italy
patrick.trentin@unitn.it

Abstract. Optimization Modulo Theories (OMT) is an extension of SMT which allows for finding models that optimize given objectives. (Partial weighted) MaxSMT–or equivalently OMT with Pseudo-Boolean objective functions, OMT+PB– is a very-relevant strict subcase of OMT. We classify existing approaches for MaxSMT or OMT+PB in two groups: MaxSAT-*based* approaches exploit the efficiency of state-of-the-art MaxSAT solvers, but they are specific-purpose and not always applicable; *OMT-based* approaches are general-purpose, but they suffer from intrinsic inefficiencies on MaxSMT/OMT+PB problems.

We identify a major source of such inefficiencies, and we address it by enhancing OMT by means of bidirectional sorting networks. We implemented this idea on top of the OptiMathSAT OMT solver. We run an extensive empirical evaluation on a variety of problems, comparing MaxSAT-based and OMT-based techniques, with and without sorting networks, implemented on top of OptiMathSAT and νZ. The results support the effectiveness of this idea, and provide interesting insights about the different approaches.

1 Introduction

Satisfiability Modulo Theories (SMT) is the problem of deciding the satisfiability of first-order formulas with respect to first-order theories [5,27] (e.g., the theory of linear arithmetic over the rationals, $\mathcal{LRA}$). In the last decade, SMT solvers –powered by very efficient Conflict-Driven-Clause-Learning (CDCL) engines for Boolean Satisfiability [16] combined with a collection of $\mathcal{T}$-Solvers, each one handling a different theory $\mathcal{T}$– have risen to be a pervasive and indispensable tool for dealing with many problems of industrial interest, e.g. formal verification of hardware and software systems, resource planning, temporal reasoning and scheduling of real-time embedded systems.

Optimization Modulo Theories (OMT) is an extension of SMT, which allows for finding models that make a given objective optimum through a combination of SMT and optimization procedures [8,9,11,12,14,15,21,28–31]. Latest advancements in OMT have further broadened its horizon by making it incremental [8,31] and by supporting objectives defined in other theories than linear arithmetic (e.g. Bit-Vectors) [8,9,17]. Moreover, OMT has been extended with the capability of handling multiple objectives at the same time either

© Springer-Verlag GmbH Germany 2017
A. Legay and T. Margaria (Eds.): TACAS 2017, Part II, LNCS 10206, pp. 231–248, 2017.
DOI: 10.1007/978-3-662-54580-5_14

independently (aka boxed optimization) or through their linear, min-max/max-min, lexicographic or Pareto combination [8,9,31].

We focus on an important strict sub-case of OMT, (partial weighted)[1] MaxSMT –or equivalently OMT with Pseudo-Boolean (PB) objective functions [25], OMT+PB– which is the problem of finding a model for an input formula which both satisfies all *hard* clauses and maximizes the cumulative weight of all *soft* clauses satisfied by the model [11,12,21]. We identify two main approaches which have been adopted in the literature (see related work). One specific-purpose approach, which we call MaxSAT-*based*, is to embed some MaxSAT engine within the SMT solver itself, and use it in combination with dedicated $\mathcal{T}$*-solvers* [3,8,9] or with SMT solvers used as blackboxes [12]. One general-purpose approach, which we call *OMT-based*, is to encode MaxSMT/OMT+PB into general OMT with linear-real-arithmetic cost functions [29].

We compare the two approaches and notice the following facts.

The MaxSAT-based approach can exploit the efficiency of state-of-the-art MaxSAT procedures and solvers. Unfortunately it suffers from some limitations that make it impractical or inapplicable in some cases. First, to the best of our knowledge, available MaxSAT engines deal with integer weights only; some applications, e.g., *(Machine) Learning Modulo Theories, LMT* [34] –a hybrid Machine Learning approach in which OMT is used as an oracle for Support Vector Machines [34]– may require the weight of soft constraints to be high-precision rational values.[2] (In this context, it is preferable not to round the weights associated with soft-clauses since it affects the accuracy of the Machine Learning approach; also multiplying all rational coefficients for their lowest common multiple of the denominators is not practical, because such values tend to become huge.) Second, a MaxSAT engine cannot be directly used when dealing with an OMT problem with multiple-independent objectives that need to be optimized at the same time [15],[3] or when the objective function is given by combinations of PB and arithmetic terms –like, e.g., for Linear Generalized Disjunctive Programming problems [26,29] or LMT problems [34].

The OMT-based approach does not suffer from the above limitations, because it exploits the infinite-precision linear-arithmetic package on the rationals of OMT solvers, and it treats PB functions as any other arithmetic functions. Nevertheless this approach may result in low performances when dealing with MaxSMT/OMT+PB problems.

We analyze the latter fact and identify a major source of inefficiency by noticing that the presence of same-weight soft clauses entails the existence of symmetries in the solution space that may lead to a combinatorial explosion of the partial truth assignments generated by the CDCL engine during the optimization search. To cope with this fact, we introduce and describe a solution

[1] Hereafter, when speaking of MaxSAT or MaxSMT, we keep "partial weighted" implicit.

[2] For example, $\frac{1799972218749879}{2251799813685248}$ is a sample weight value from problems in [34].

[3] One could run a MaxSAT-based search separately on each objective, but doing this he/she would loose the benefits of boxed optimization, see [8,15,31].

based on (bidirectional) sorting networks [1,4,32]. We implemented this idea within the OptiMathSAT OMT solver [30].

We run an empirical evaluation on a large amount of problems comparing MaxSAT-based and OMT-based techniques, with and without sorting networks, implemented on top of OptiMathSAT [30] and νZ [9]. The results are summarized as follows.

(a) Comparing MaxSAT-based wrt. OMT-based approaches on problems where the former are applicable, it turns out that the former provide much better performances, in particular when adopting the maximum-resolution [8,18] MaxSAT engine.
(b) Evaluating the benefits of bidirectional sorting-network encodings, it turns out that they improve significantly the performances of OMT-based approaches, and often also of MaxSAT-based ones.
(c) Comparing νZ and OptiMathSAT, it turns out that the former performed better on MaxSAT-based approaches, whilst the latter performed seomtimes equivalently and sometimes significantly better on OMT-based ones, in particular when enhanced by the sorting-network encoding.

Related Work. The idea of MaxSMT and of optimization in SMT was first introduced by Nieuwenhuis & Oliveras [21], who presented a general logical framework of "SMT with progressively stronger theories" (e.g., where the theory is progressively strengthened by every new approximation of the minimum cost), and presented implementations for MaxSMT based on this framework. Cimatti et al. [11] introduced the notion of "Theory of Costs" $\mathcal{C}$ to handle Pseudo-Boolean (PB) cost functions and constraints by an ad-hoc "$\mathcal{C}$-solver" in the standard lazy SMT schema, and implemented a variant of MathSAT tool able to handle SMT with PB constraints and to minimize PB cost functions. Cimatti et al. [12] presented a "modular" approach for MaxSMT, combining a lazy SMT solver with a MaxSAT solver, where the SMT solver is used as an oracle generating $\mathcal{T}$-lemmas that are then learned by the MaxSAT solver so as to progressively narrow the search space toward the optimal solution.

Sebastiani and Tomasi [28,29] introduced a wider notion of optimization in SMT, namely *Optimization Modulo Theories (OMT) with $\mathcal{LRA}$ cost functions*, OMT($\mathcal{LRA} \cup \mathcal{T}$), which allows for finding models minimizing some $\mathcal{LRA}$ cost term –$\mathcal{T}$ being some (possibly empty) stably-infinite theory s.t. $\mathcal{T}$ and $\mathcal{LRA}$ are signature-disjoint– and presented novel OMT($\mathcal{LRA} \cup \mathcal{T}$) tools which combine standard SMT with LP minimization techniques. ($\mathcal{T}$ can also be a combination of Theories $\bigcup_i \mathcal{T}_i$.) Eventually, OMT($\mathcal{LRA} \cup \mathcal{T}$) has been extended so that to handle costs on the integers,incremental OMT, multi-objective, and lexicographic OMT and Pareto-optimality [8,9,14,15,30,31]. To the best of our knowledge only four OMT solvers are currently implemented: bclt [14], νZ (aka Z3Opt) [8,9], OptiMathSAT [30,31], and Symba [15]. Remarkably, bclt, νZ and OptiMathSAT currently implement also specialized procedures for MaxSMT, leveraging to SMT level state-of-the-art MaxSAT procedures; in addition, νZ features a Pseudo-Boolean $\mathcal{T}$*-solver* which can generate sorting

circuits on demand for Pseudo-Boolean inequalities featuring sums with small coefficients when a Pseudo-Boolean inequality is used some times for unit propagation/conflicts [7,9].

Content. The paper is structured as follows. Section 2 briefly reviews the background; Sect. 3 describes the source of inefficiency arising when MaxSMT is encoded in OMT as in [29]; Sect. 4 illustrates a possible solution based on bidirectional sorting networks; in Sect. 5 we provide empirical evidence of the benefits of this approach on two applications of OMT interest. Section 6 provides some conclusions with some considerations on the future work.

2 Background

We assume the reader is familiar with the main theoretical and algorithmic concepts in SAT and SMT solving (see [5,16]). Optimization Modulo Theories (OMT) is an extension of SMT which addresses the problem of finding a model for an input formula φ which is optimal wrt. some objective function obj [21,28]. The basic minimization scheme implemented in state-of-the-art OMT solvers, known as *linear-search* scheme [21,28], requires solving an SMT problem with a solution space that is progressively tightened by means of unit linear constraints in the form $\neg(ub_i \leq obj)$, where ub_i is the value of obj that corresponds to the optimum model of the most-recently found truth assignment μ_i s.t. $\mu_i \models \varphi$. The ub_i value is computed by means of a specialized optimization procedure embedded within the $\mathcal{T}$*-solver* which, taken as input a pair $\langle \mu, obj \rangle$, returns the optimal value ub of obj for such μ. The OMT search terminates when such procedure finds that obj is unbounded or when the SMT search is UNSAT, in which case the latest value of obj (if any) and its associated model M_i is returned as optimal solution value. (Alternatively, binary-search schemes can also be used [28,29].)

An important subcase of OMT is that of MaxSMT, which is a pair $\langle \varphi_h, \varphi_s \rangle$, where φ_h denotes the set of "hard" $\mathcal{T}$-clauses, φ_s is a set of positive-weighted "soft" $\mathcal{T}$-clauses, and the goal is to find the maximum-weight set of $\mathcal{T}$-clauses ψ_s, $\psi_s \subseteq \varphi_s$, s.t. $\varphi_h \cup \psi_s$ is $\mathcal{T}$-satisfiable [3,11,12,21]. As described in [29], MaxSMT $\langle \varphi_h, \varphi_s \rangle$ can be encoded into a general OMT problem with a Pseudo-Boolean objective: first introduce a fresh Boolean variable A_i for each soft-constraint $C_i \in \varphi_s$ as follows

$$\varphi^* \stackrel{\text{def}}{=} \varphi_h \cup \bigcup_{C_i \in \varphi_s} \{(A_i \vee C_i)\}; \;\; obj \stackrel{\text{def}}{=} \sum_{C_i \in \varphi_s} w_i A_i \tag{1}$$

and then encode the problem into OMT as a pair $\langle \varphi, obj \rangle$ where φ is defined as

$$\varphi \stackrel{\text{def}}{=} \varphi^* \wedge \textstyle\bigwedge_i ((\neg A_i \vee (x_i = w_i)) \wedge (A_i \vee (x_i = 0))) \wedge \tag{2}$$

$$\textstyle\bigwedge_i ((0 \leq x_i) \wedge (x_i \leq w_i)) \wedge \tag{3}$$

$$(obj = \textstyle\sum_i x_i), \qquad x_i, \; obj \; fresh. \tag{4}$$

Notice that, although redundant from a logical perspective, the constraints in (3) serve the important purpose of allowing early-pruning calls to the $\mathcal{LRA}$-Solver (see [5]) to detect a possible $\mathcal{LRA}$ inconsistency among the current partial truth assignment over variables A_i and linear cuts in the form $\neg(ub \leq obj)$ that are pushed on the formula stack by the OMT solver during the minimization of obj. To this extent, the presence of such constraints improves performance significantly.

3 Problems with OMT-based Approaches

Consider first the case of a MaxSMT-derived OMT problem as in (1) s.t. all weights are identical, that is: let $\langle \varphi, obj \rangle$ be an OMT problem, where $obj = \sum_{i=1}^{n} w \cdot A_i$, where the A_is are Boolean variables, and let μ be a satisfiable truth assignment found by the OMT solver during the minimization of obj. Given $A_T = \{A_i | \mu \models A_i\}$ and $k = |A_T|$, then the upper bound value of obj in μ is $ub = w \cdot k$. As described in [28,29], the OMT solver adds a unit clause in the form $\neg(ub \leq obj)$ in order to (1) remove the current truth assignment μ from the feasible search space and (2) seek for another μ' which improves the current upper-bound value ub. Importantly, the unit clause $\neg(ub \leq obj)$ does not only prune the current truth assignment μ from the feasible search space, but it also makes inconsistent any other (partial) truth assignment μ' which sets exactly k (or more) A_i variables to True. Thus, each new unit clause in this form prunes $\gamma = \binom{n}{k}$ truth assignments from the search space, where γ is the number of possible permutations of μ over the variables A_i. A dual case occurs when some lower-bound unit clause $\neg(obj \leq lb)$ is learned (e.g., in a binary-search step, see [28]).

Unfortunately, the inconsistency of a truth assignment μ' which sets exactly k variables to True wrt. a unit clause $\neg(ub \leq obj)$, where $ub = w \cdot k$, cannot be determined by simple Boolean Constraint Propagation (BCP). In fact, $\neg(ub \leq obj)$ being a $\mathcal{LRA}$ term, the CDCL engine is totally oblivious to this inconsistency until when the $\mathcal{T}$-*solver* for linear rational arithmetic ($\mathcal{LRA}$-Solver) is invoked, and a conflict clause is generated. Therefore, since the $\mathcal{LRA}$-Solver is much more resource-demanding than BCP and it is invoked less often, it is clear that the performance of an OMT solver can be negatively affected when dealing with this kind of objectives.

Example 1. Figure 1 shows a toy example of OMT search execution over the pair $\langle \varphi, obj \rangle$, where φ is some SMT formula and $obj \stackrel{\text{def}}{=} \sum_{i=1}^{4} A_i$ (i.e., $w_i = 1$ for every i). We assume the problem has been encoded as in (2)–(4), so that the truth assignment $\mu_0 \stackrel{\text{def}}{=} \cup_{i=1}^{4} \{(0 \leq x_i), (x_i \leq 1)\} \cup \{(obj = \sum_{i=1}^{4} x_i)\}$ is immediately generated by BCP, and is part of all truth assignments generated in the search. In the first branch (left) a truth assignment $\mu \stackrel{\text{def}}{=} \mu_0 \cup \{A_1, (x_1 = 1), A_2, (x_2 = 1), \neg A_3, (x_3 = 0), \neg A_4, (x_4 = 0)\}$ is found s.t. $obj = 2$, resulting from the decisions A_1, A_2, $\neg A_3$ and $\neg A_4$. Then the unit clause $\neg(2 \leq obj)$ is learned and the Boolean search is restarted in order to find an improved solution. In the second branch (center) A_1 and A_2 are decided, forcing by BCP the assignment

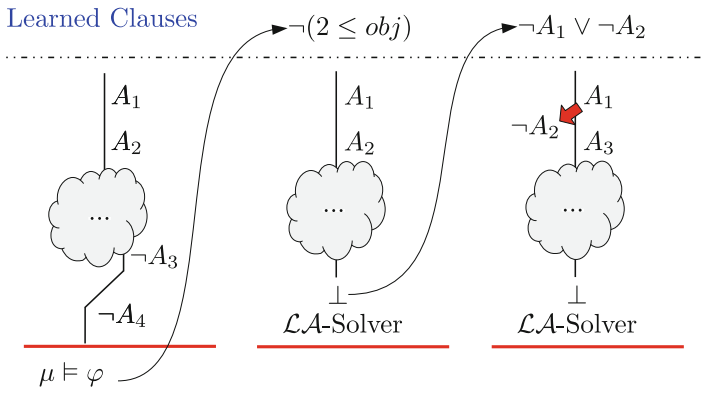


Fig. 1. A simple example of OMT search.

$\mu' \stackrel{\text{def}}{=} \mu_0 \cup \{\neg(2 \leq obj), A_1, (x_1 = 1), A_2, (x_2 = 1)\}$ which is $\mathcal{LRA}$-inconsistent. However, it takes a (possibly-expensive) intermediate call to the $\mathcal{LRA}$-Solver to reveal such an inconsistency.[4] If so, a new conflict clause $\neg A_1 \vee \neg A_2$ is learned, forcing the solver to back-jump and toggle the value of A_2 (right). The search continues with the new decision A_3, which is again $\mathcal{LRA}$ inconsistent, causing a new conflict clause as before, and so on. In this way, the solver might uselessly enumerate and check all the up-to $\binom{4}{2}$ assignments that assign two A_i's to true and are consistent with φ, even though they are intrinsically incompatible with $\neg(2 \leq obj)$. ◇

The performance issue identified with the previous case example can be generalized to any objective obj in which groups of A_i's share the same weights:

$$obj = \tau_1 + \ldots + \tau_m, \tag{5}$$

$$\textstyle\bigwedge_{j=1}^{m} ((\tau_j = w_j \cdot \sum_{i=1}^{k_j} A_{ji}) \ \wedge \ (0 \leq \tau_j) \wedge (\tau_j \leq w_j \cdot k_j)), \tag{6}$$

where the logically-redundant constraints $(0 \leq \tau_j) \wedge (\tau_j \leq w_j \cdot k_j)$ are added for the same reason as with (3).

4 Combining OMT with Sorting Networks

Notationally, the symbols $\top, \bot, *$ denote respectively "true", "false" and "unassigned". We represent truth assignment as sets (or conjunctions) of literals s.t. a positive [resp. negative] literal denotes the fact that the corresponding atom is assigned to $\top$ [resp. $\bot$]. Given a Boolean formula φ and two truth assignments μ, η on the atoms in φ, "$\langle \varphi, \mu \rangle \vdash_{\mathsf{bcp}} \eta$" denotes the fact that all literals in η are inferred by BCP on φ if all literals in μ are asserted. (Notice that "$\langle \varphi, \mu \rangle \vdash_{\mathsf{bcp}} \eta$" is stronger than "$\varphi \wedge \mu \models \eta$".)

[4] The fact that such call is actually performed depends on the early-pruning strategy implemented in the OMT solver; alternatively, a possibly-expensive $\mathcal{T}$-propagation step on the previous $\mathcal{LRA}$-Solver call has a similar effect. (See e.g. [5,27].)

When dealing with MaxSMT and OMT with PB objectives in the form

$$obj = w \cdot \sum_{i=1}^{n} A_i \tag{7}$$

a solution for improving search efficiency is to reduce the dependency on the expensive $\mathcal{LRA}$-Solver by better exploiting BCP with the aid of Boolean *bidirectional sorting networks.*

Definition 1. *Let* $\mathsf{SN}[\underline{A}, \underline{B}]$ *be a CNF Boolean formula on* n *input Boolean variables* $\underline{A} \stackrel{def}{=} \{A_1, \ldots, A_n\}$ *and* n *output Boolean variables* $\underline{B} \stackrel{def}{=} \{B_1, \ldots, B_n\}$*, possibly involving also auxiliary Boolean variables which are not mentioned.*

We say that $\mathsf{SN}[\underline{A}, \underline{B}]$ *is a* **bidirectional sorting network** *if and only if, for every* m *and* k *s.t.* $n \geq m \geq k \geq 0$ *and for every partial truth assignment* μ *s.t.* μ *assigns exactly* k *input variables* A_i *to* $\top$ *and* $n - m$ *variables* A_i *to* $\bot$*:*

$$\langle \mathsf{SN}[\underline{A}, \underline{B}], \mu \rangle \vdash_{\mathsf{bcp}} \{ \ B_1, \ldots, \ B_k\}, \tag{8}$$

$$\langle \mathsf{SN}[\underline{A}, \underline{B}], \mu \rangle \vdash_{\mathsf{bcp}} \{\neg B_{m+1}, \ldots, \neg B_n\}. \tag{9}$$

$$\langle \mathsf{SN}[\underline{A}, \underline{B}], \mu \cup \{\neg B_{k+1}\} \rangle \vdash_{\mathsf{bcp}} \{\neg A_i \ s.t. \ A_i \ unassigned \ in \ \mu\}, \tag{10}$$

$$\langle \mathsf{SN}[\underline{A}, \underline{B}], \mu \cup \{ \ B_m\} \rangle \vdash_{\mathsf{bcp}} \{A_i \ s.t. \ A_i \ unassigned \ in \ \mu\}. \tag{11}$$

The schema of a bidirectional sorting network is depicted in Fig. 2.

(8)–(9) state that the output values $\underline{B}$ of $\mathsf{SN}[\underline{A}, \underline{B}]$ are propagated from the inputs $\underline{A}$ via BCP. (10)–(11) describe how assigning output variables $\underline{B}$ propagates back to input variables $\underline{A}$: (10) states that, when k A_i's are true and B_{k+1} is false, then all other A_i's are forced to be false by BCP; dually, (11) states that, when $n - m$ A_i's are false and B_m is true, then all other A_i's are forced to be true by BCP. (If any of the above BCP assignments conflicts with some previous assignment, a conflict is produced.)

Given an OMT problem $\langle \varphi, obj \rangle$, where obj is as in (7), and a Boolean formula $\mathsf{SN}[\underline{A}, \underline{B}]$ encoding a bidirectional sorting network relation as in Definition 1, we extend φ in (2)–(4) as follows:

$$\varphi' = \varphi \wedge \mathsf{SN}[\underline{A}, \underline{B}] \wedge \bigwedge_{i=1}^{n} \begin{cases} (\neg B_i \vee (i \cdot w \leq obj)) \wedge \\ (B_i \vee (obj \leq (i-1) \cdot w)) \wedge \\ (\neg(i \cdot w \leq obj) \vee \neg(obj \leq (i-1) \cdot w)) \end{cases} \tag{12}$$

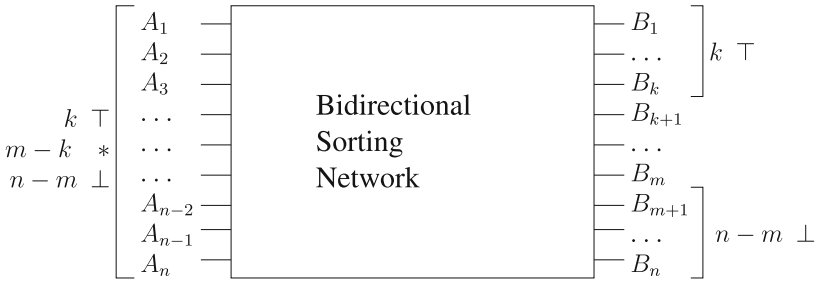


Fig. 2. The basic schema of a bidirectional sorting network.

and optimize obj over φ'. Notice here that the third line in Eq. 12 is $\mathcal{LRA}$-valid, but it allows for implying the negation of $(obj \leq (i-1)\cdot w)$ from $(i \cdot w \leq obj)$ (and vice versa) directly by BCP, without any call to the $\mathcal{LRA}$-Solver.

Consider (8)–(9) and assume that μ assigns k A_is to $\top$ and $n-m$ to $\bot$ as in Definition 1. Then (8) with (12) forces the unit-propagation of $B_1, \ldots, B_k$, and then, among others, of $(k \cdot w \leq obj)$, while (9) with (12) forces the unit-propagation of $\neg B_{m+1}, \ldots, \neg B_n$, and then, among others, of $(obj \leq m \cdot w)$. This automatically restricts the range of obj to $[k \cdot w, m \cdot w]$, obtaining the same effect as (2)–(4).

The benefits of the usage of $\mathsf{SN}[\underline{A}, \underline{B}]$ are due to both (10) and (11). When the optimization search finds a new minimum $k \cdot w$ and a unit clause in the form $\neg(k \cdot w \leq obj)$ is learned (see e.g. [28]) and $\neg B_k$ is unit-propagated on (12), then as soon as $k-1$ A_is are set to True, all the remaining $n-k+1$ A_is are set to False by BCP (10). A dual case occurs when some lower-bound unit clause $\neg(obj \leq k \cdot w)$ is learned (e.g., in a binary-search step [28]) and B_{k+1} is unit-propagated on (12): as soon as $n-k-1$ A_is are set to False, then all the remaining $k+1$ A_is are set to True by BCP (11).

Example 2. Figure 3 considers the same scenario as in Example 1, in which we extend the encoding with a bidirectional sorting-network relation as in (12). The behaviour is identical to that of Example 1 until the assignment μ is generated, the unit clause $\neg(2 \leq obj)$, and the procedure backtracks for the first time (Fig. 3 left). This causes the unit-propagation of $\neg B_2$ on (12). As soon as A_1 is picked as new decision, $\neg A_2, \neg A_3, \neg A_4$ are unit propagated (10), saving up to $\binom{4}{2}$ (expensive) calls to the $\mathcal{LRA}$-Solver (Fig. 3 center). Then $\neg(1 \leq obj)$ is learned, and the search proceeds (Fig. 3 right). ◇

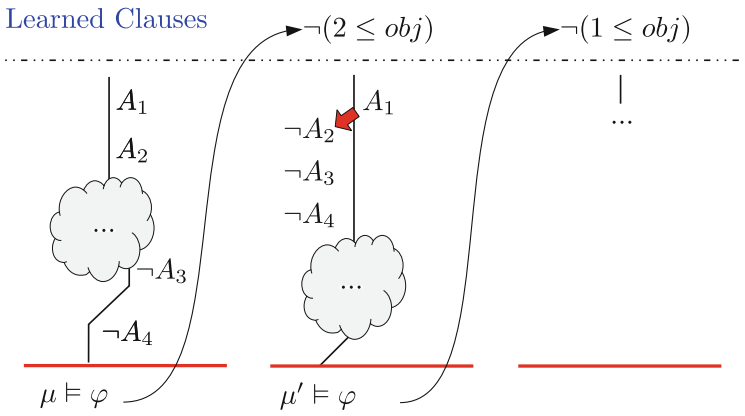


Fig. 3. An example of OMT search with sorting networks.

We generalize this approach to deal with the general objectives as in (5)–(6). In this case a separate sorting circuit is generated for each term τ_j, and constraints in the form

$$\bigwedge_{j=1}^{m} \bigwedge_{i=1}^{k_j} (\neg(w_j \cdot i \leq obj) \rightarrow \neg(w_j \cdot i \leq \tau_j)), \tag{13}$$

are added to ensure that the circuit is activated by BCP.

4.1 Bidirectional Sorting Networks

Unlike the usage of sorting networks in other contexts, which consider only (8) and (10) as relevant properties (e.g. [32]), we are interested in sorting networks which propagate both $\top$ and $\bot$ values in both directions (i.e., which comply with all properties (8)–(11)). To this extent, we have considered two encodings: the sequential counter encoding in [32], which we have extended to comply with all properties (8)–(11), and the cardinality network encoding in [1,4].

Bidirectional Sequential Counter Encoding. The sequential counter encoding $LT_{SEQ}^{n,k}$ for $\leq k(A_1, \ldots, A_n)$ presented in [32] consists of $O(k \cdot n)$ clauses and variables and complies with (8) and (10). The circuit is given by the composition of n sub-circuits, each of which computes $S_i = \sum_{j=1}^{i} A_j$, represented in unary form with the bits $S_{i,j}$, i.e., $S_{i,j} = \top$ if $\sum_{r=1}^{i} A_r \geq j$, so that $B_j \stackrel{\text{def}}{=} S_{n,j}$, $j \in [1 \ldots n]$. The (CNF version of the)[5] following formula is the encoding of $LT_{SEQ}^{n,k}$ presented in [32], with $k \stackrel{\text{def}}{=} n$:

$$(A_1 \rightarrow S_{1,1}) \wedge \bigwedge_{i=2}^{n} \{((A_i \vee S_{i-1,1}) \rightarrow S_{i,1})\} \wedge \tag{14}$$

$$\bigwedge_{i=2}^{n} \{(\neg A_i \vee \neg S_{i-1,n})\} \wedge \bigwedge_{j=2}^{n} \{(\neg S_{1,j})\} \wedge \tag{15}$$

$$\bigwedge_{i,j=2}^{n} \{(((A_i \wedge S_{i-1,j-1}) \vee S_{i-1,j}) \rightarrow S_{i,j})\} \tag{16}$$

Notice that, in order to reduce the size of the encoding, in (14)–(16) only right implications "$\rightarrow$" were used to encode each gate in the Boolean sorting circuit [32], so that (14)–(16) complies with (8) and (10) but not with (9) and (11). To cope with this fact, we have added the following part, which reintroduces the left implications "$\leftarrow$" of the encoding of each gate in (14) and (16), making it compliant also with (9) and (11):

$$(A_1 \leftarrow S_{1,1}) \wedge \bigwedge_{i=2}^{n} \{((A_i \vee S_{i-1,1}) \leftarrow S_{i,1})\} \wedge \tag{17}$$

$$\bigwedge_{i,j=2}^{n} (((A_i \wedge S_{i-1,j-1}) \vee S_{i-1,j}) \leftarrow S_{i,j}). \tag{18}$$

Bidirectional Cardinality Network Encoding. The cardinality network encoding presented in [1,4,13], based on the underlying sorting scheme of the well-known *merge-sort* algorithm, has complexity $O(n \log^2 k)$ in the number of clauses and

[5] Here (14)–(18) are written as implications to emphasize the directionality of the encodings.

variables. Due to space limitations, we refer the reader to [1,4] for the encoding of cardinality networks we used in our own work. Notice that, differently than in the previous case, this sorting network propagates both $\top$ and $\bot$ values in both directions (i.e., it complies with all properties (8)–(11) [1,4] and it is thus suitable to be used within OMT without modifications.

Both of the previous encodings are istantiated assuming $k = n$, since the sorting network is generated prior to starting the search. Therefo re, the cardinality network circuit looks more appealing than the sequential counter encoding due to its lower complexity in terms of clauses and variables employed.

5 Experimental Evaluation

We extended OptiMathSAT with a novel internal preprocessing step, which automatically augments the input formula with a sorting network circuit of choice between the bidirectional sequential counter and the cardinality network, as described in Sect. 4. To complete our comparison, we also implemented in OptiMathSAT two MaxSAT-based approaches, the max-resolution approach implemented in νZ [8,18] and (for MaxSMT only) the lemma-lifting approach of [12], using Maxino [2] as external MaxSAT solver.

Here we present an extensive empirical evaluation of various MaxSAT-based and OMT-based techniques in OptiMathSAT [23,30] and νZ [9,22]. Overall, we considered >20,000 OMT problems and run >270,000 job pairs. The problems were produced either by CGM-Tool [10] from optimization of Constrained Goal Models [19,20] (a modeling and automated-reasoning tool for requirement engineering) or by PyLMT [24] from (Machine) Learning Modulo Theories [34]. We partition these problems into two distinct categories. In Sect. 5.1 we analyze problems which are *solvable by* MaxSAT-*based approaches*, like those with PB objective functions or their lexicographic combination, so that to allow both νZ and OptiMathSAT to use their MaxSAT-specific max-resolution engines (plus others). In Sect. 5.2 we analyze problems which *cannot* be solved by MaxSAT-based approaches, because the objective functions involve some non-PB components, forcing to restrict to OMT-based approaches only.

The goal of this empirical evaluation is manyfold:

(i) compare the performance of MaxSAT-based approaches wrt. OMT-based ones, on the kind of OMT problems where the former are applicable;
(ii) evaluate the benefits of sorting-network encodings with OMT-based approaches (and also with MaxSAT-based ones);
(iii) compare the performances of OptiMathSAT with those of νZ.

For goals (i) and (ii) we used the following configurations of OptiMathSAT.

OMT-based: standard, enriched with the bidirectional sequential-counter and cardinality sorting network;
MaxSAT-based: the above-mentioned max-resolution implementation, with and without the cardinality sorting network, and lemma-lifting (for pure MaxSMT only).

For goal (iii) we also used the following configurations of νZ.[6]

OMT-based: standard (encoded as in (2)–(4)).
MaxSAT-based: using alternatively the internal implementations of the max-resolution [8,18] and Wmax [21] procedures.

Each job pair was run on one of two identical Ubuntu Linux machines featuring *8-core Intel-Xeon@2.20 GHz* CPU, 64 GB of ram and kernel 3.8-0-29. Importantly, we verified that all tools/configurations under test agreed on the results on all problems when terminating within the timeout. (The timeout varies with the benchmark sets, see Sects. 5.1 and 5.2.) All benchmarks, as well as our experimental results and all the tools which are necessary to reproduce the results, are available [33].

5.1 Problems Suitable for MaxSAT-Based Approaches

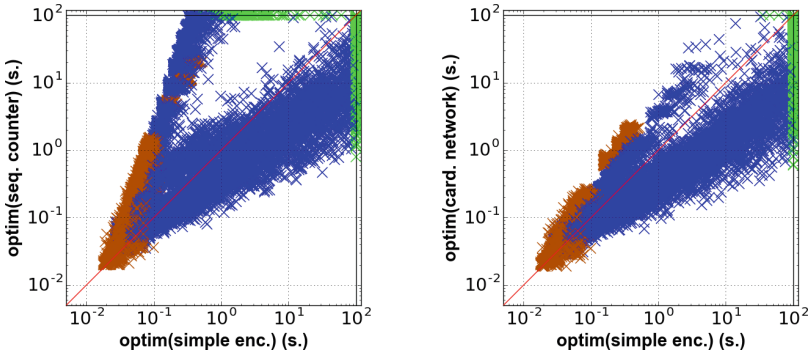
Test Set #1: CGMs with Lexicographic PB Optimization. In our first experiment we consider the set of all problems produced by CGM-Tool [10] in the experimental evaluation in [19]. They consist of 18996 automatically-generated formulas which encode the problem of computing the lexicographically-optimum realization of a constrained goal model [19], according to a prioritized list of (up to) three objectives $\langle obj_1, obj_2, obj_3 \rangle$. A solution *optimizes lexicographically* $\langle obj_1, \ldots, obj_k \rangle$ if it optimizes obj_1 and, if more than one such obj_1-optimum solutions exists, it also optimizes $obj_2, \ldots$, and so on; both OMT-based and MaxSAT-based techniques handle lexicographic optimization, by optimizing $obj_1, obj_2, \ldots$ in order, fixing the value of each obj_i to its optimum as soon as it is found [8,9,30,31]. In this experiment, we set the timeout at 100 s. The results are reported in Fig. 4 (top and middle).

As far as OptiMathSAT (OMT-based) is concerned, extending the input formula with either of the sorting networks increases the number of benchmarks solved within the timeout. Notably, the cardinality network encoding –which has the lowest complexity– scores the best both in terms of number of solved benchmarks and solving time. On the other hand, the sequential counter network is affected by a significant performance hit on a number of benchmarks, as it is witnessed by the left scatter plot in Fig. 4. This not only affects unsatisfiable benchmarks, for which using sorting networks appears to be not beneficial in general, but also satisfiable ones.

A possible strategy for overcoming this performance issue is to reduce the memory footprint determined by the generation of the sorting network circuit.

[6] Notice that, unlike OptiMathSAT, νZ selects automatically its presumably-best configuration for a given input problem. In particular, when MaxSMT-encodable problems are fed to νZ –like, e.g., those in Sect. 5.1– νZ forces automatically the choice of the MaxSAT-based configuration, allowing the user only the choice of the MaxSAT algorithm. Thus we could not test νZ also with OMT-based configuration for the problems in Sect. 5.1. Alternatively, we should have disguised the input problem, with the risk of affecting the significance or the result.

tool, configuration & encoding	inst.	term.	timeout	time (s.)
OptiMathSAT (OMT-based)	18996	16316	2680	48832
OptiMathSAT (OMT-based + seq. counter)	18996	16929	2067	90080
OptiMathSAT (OMT-based + card. network)	18996	**17191**	1805	39215
OptiMathSAT (MaxSAT-based w. maxres)	18996	17933	1063	24369
OptiMathSAT (MaxSAT-based w. maxres + seq. counter)	18996	18180	816	49088
OptiMathSAT (MaxSAT-based w. maxres + card. netw.)	18996	**18197**	799	26489
νZ (MaxSAT-based w. maxres)	18996	**18996**	0	1640



partitioned sequential-counter encoding				
vars	inst.	term.	timeout	time (s.)
∞	18996	16929	2067	90080
10	18996	17033	1963	39035
15	18996	17061	1935	39264
20	18996	**17152**	1844	43730

partitioned cardinality-network encoding				
vars	inst.	term.	timeout	time (s.)
∞	18996	**17191**	1805	39215
10	18996	17058	1938	36636
15	18996	17133	1863	37246
20	18996	17190	1806	39492

Fig. 4. [Top, table] Results of various solvers, configurations and encodings on all the problems encoding CGM optimization with lexicographic PB optimization of [19,20]. (Values in **boldface** denote the best performance of each category; values in *blue* denote the absolute best performance.) [Middle, scatterplots]. Pairwise comparison on OptiMathSAT (OMT-based) with/out sequential-counter encoding (left) and with/out cardinality-network encoding (right). (*Brown* points denote unsatisfiable benchmarks, *blue* denote satisfiable ones and *green* ones represent timeouts.) [Bottom, tables] Effect of splitting the PB sums into chunks of maximum variable number (no split, 10, 15, 20 variables) with the sequential-counter encoding (left) and the cardinality-network encoding (right). (Color figure online)

This can be easily achieved by splitting each Pseudo-Boolean sum in smaller sized chunks and generating a separate sorting circuit for each splice. The result of applying this enhancement, using chunks of increasing size, is shown in Fig. 4 (bottom). The data suggest that the sequential counter encoding can benefit from this simple heuristic, but it does not reach the performances of the cardinality network, which are not affected by this strategy. (In next experiments this strategy will be no more considered.)

As far as OptiMathSAT (MaxSAT-based) is concerned, we notice that it significantly outperforms all OMT-based techniques. Remarkably, extending the input formula with the sorting networks improves the performance also of this configuration.

tool, configuration & encoding	inst.	term.	timeout	time (s.)
OptiMathSAT (OMT-based)	2499	1794	705	11178
OptiMathSAT (OMT-based + seq. counter)	2499	**2451**	48	18033
OptiMathSAT (OMT-based + card. network)	2499	2186	313	10633
OptiMathSAT (MaxSAT-based w. maxres)	2499	**2499**	0	**128**
OptiMathSAT (MaxSAT-based w. maxres + seq. counter)	2499	2499	0	1638
OptiMathSAT (MaxSAT-based w. maxres + card. netw.)	2499	2499	0	257
OptiMathSAT (lemma-lifting w. Maxino)	2499	2497	2	343
νZ (MaxSAT-based w. maxres)	2499	**2499**	0	**119**
νZ (MaxSAT-based w. wmax)	2499	1799	733	10549

Fig. 5. Results of various solvers, configurations and encodings on CGM-encoding problems of [19,20] with single-objective weight-1.

As far as νZ (MaxSAT-based) is concerned, we notice that when using the max-resolution algorithm it outperforms all other techniques by solving all problems.

Test Set #2: CGMs with Weight-1 PB Optimization. Our second experiment is a variant of the previous one, in which we consider only single-objective optimizations and we fix all weights to 1, so that each problem is encoded as a plain un-weighted MaxSMT problem. We set the timeout to 100 s. The results are reported in Fig. 5.

As far as OptiMathSAT (OMT-based) is concerned, extending the input formula with either of the sorting networks increases the number of benchmarks solved within the timeout. Surprisingly, this time the sequential counter network performs significantly better than the cardinality network, despite its bigger size. (We do not have a clear explanation of this fact.)

As far as OptiMathSAT (MaxSAT-based) is concerned, we notice that it significantly outperforms all OMT-based techniques, solving all problems. Extending the input formula with the cardinality networks slightly worsens the performances. Also the lemma-lifting techniques outperforms all OMT-based techniques, solving only two problem less than the previous MaxSAT-based techniques.

As far as νZ (MaxSAT-based) is concerned, we notice that using the max-resolution MaxSAT algorithm it is the best scorer, although the differences wrt. OptiMathSAT (MaxSAT-based) are negligible, whilst by using the `wmax` engine the performances decrease drastically.

5.2 Problems Unsuitable for MaxSAT-Based Approaches

Here we present a couple of test sets which cannot be supported by any MaxSAT-based technique in OptiMathSAT or νZ and, to the best of our knowledge, no encoding of these problem into MaxSMT has ever been conceived. Thus the solution is restricted to OMT-based techniques. (To this extent,

with OptiMathSAT we have used the linear-search strategy rather than the default adaptive linear/binary one to better compare with the linear strategy adopted by νZ.)

Test Set #3: CGMs with Max-Min PB Optimization. In our third experiment we consider another variant of the problems in Test Set #1, in which the three PB/MaxSMT objectives $\langle obj_1, obj_2, obj_3 \rangle$ are subject to a max-min combination: each objective obj_j is normalized so that its range equals $[0, 1]$ (i.e., it is divided by $\sum_i w_{ji}$), then $\bigwedge_{j=1}^{3}(obj_j \leq obj)$ s.t. obj is a fresh $\mathcal{LRA}$ variable is added to the main formula, and the solver is asked to find a solution making obj minimum (see [30]). Notice that max-min optimization guarantees a sort of "fairness" among the objectives $obj_1, \ldots, obj_3$. Since the problem is more complex than the previous ones and the most-efficient MaxSAT-based techniques are not applicable, we increased the timeout to 300 s. The results are shown in Fig. 6. (Unlike with Fig. 4, since the difference in performance between OptiMathSAT with the two sorting networks is minor, here and in Fig. 7 we have dropped the scatterplot with the sequential-counter encoding and we introduced one comparing with νZ instead.)

Looking at the table and at the scatterplot on the left, we notice that enhancing the OMT-based technique of OptiMathSAT by adding the cardinality networks improves significantly the performances. Also, looking at the table and at the scatterplot on the right, we notice that OMT-based technique of OptiMathSAT, with the help of sorting networks, performs equivalently or slightly better than that of νZ.

tool, configuration & encoding	inst.	term.	timeout	time (s.)
OptiMathSAT (OMT-based)	2399	2340	59	20841
OptiMathSAT (OMT-based + seq. counter)	2399	2394	5	9511
OptiMathSAT (OMT-based + card. network)	2399	**2395**	4	8275
νZ (OMT-based)	2399	2390	9	8076

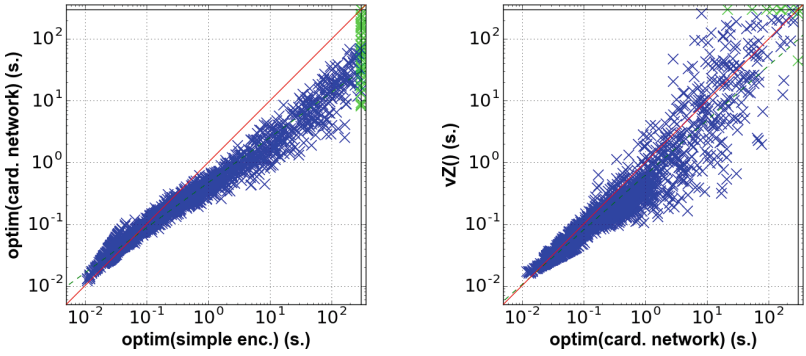


Fig. 6. [Table:] results of various solvers with OMT-based configurations on CGMencoding problems of [19,20] with max-min objective functions. [Left scatterplot:] OptiMathSAT + card. network vs. plain OptiMathSAT. [Right scatterplot:] νZ vs. OptiMathSAT + card. network.

Test Set #4: LMT with Mixed Complex Objective Functions. In our fourth experiment we consider a set of 500 problems taken from PyLMT [24], a tool for Structured Learning Modulo Theories [34] which uses OptiMathSAT as back-end oracle for performing inference in the context of machine learning in hybrid domains. The objective functions *obj* are complex combinations of PB functions in the form:

$$obj \stackrel{\text{def}}{=} \sum_j w_j \cdot B_j + cover - \sum_k w_k \cdot C_k - |K - cover|, \tag{19}$$

$$s.t.\ cover \stackrel{\text{def}}{=} \textstyle\sum_i w_i A_i, \tag{20}$$

A_i, B_j, C_k being Boolean atoms, w_i, v_j, z_k, K being rational constants. We imposed a timeout of 600 s. The results are presented in Fig. 7.

Looking at the table and at the scatterplot on the left, we notice that enhancing the OMT-based technique of OptiMathSAT by adding the cardinality networks improves the performances, although this time the improvement is not dramatic. (We believe this is due that the values of the weights w_i, v_j, z_k, K are very heterogeneous, not many weights share the same value.) Also, looking at the table and at the scatterplot on the right, we notice that OMT-based technique of OptiMathSAT performs significantly better than that of νZ, even without the help of sorting networks.

tool, configuration & encoding	inst.	term.	timeout	time (s.)
OptiMathSAT (OMT-based)	500	421	79	2607
OptiMathSAT (OMT-based + seq. counter)	500	441	59	6381
OptiMathSAT (OMT-based + card. network)	500	**442**	58	6189
νZ (OMT-based)	500	406	94	2120

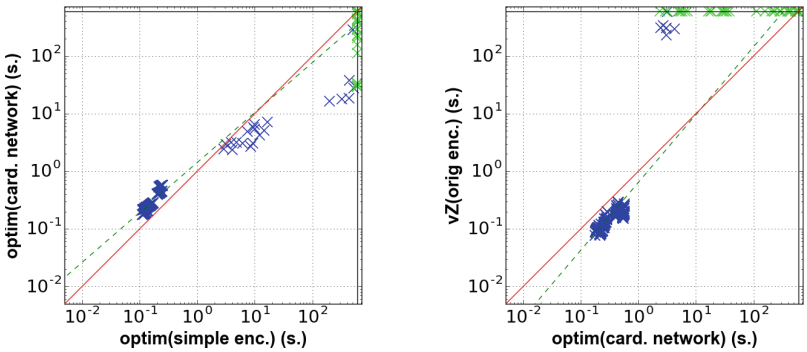


Fig. 7. [Table:] results of various solvers with OMT-based configurations on LMT-encoding problems of [34] with complex objective functions. [Left scatterplot:] OptiMathSAT + card. network vs. plain OptiMathSAT. [Right scatterplot:] νZ vs. OptiMathSAT + card. network.

Discussion. We summarize the results as follows.

(a) When applicable, MaxSAT-based approaches performed much better than OMT-based ones, in particular when adopting Maximum-Resolution as MaxSAT engine.
(b) Bidirectional sorting-network encodings improved significantly the performances of OMT-based approaches, and often also of MaxSAT-based ones.
(c) νZ performed better than OptiMathSAT on MaxSAT-based approaches, whilst the latter performed sometimes similarly and sometimes significantly better on OMT-based ones, in particular when enhanced by the sorting-network encodings.

6 Conclusion and Future Work

MaxSMT and OMT with Pseudo-Boolean objective functions are important sub-cases of OMT, for which specialized techniques have been developed over the years, in particular exploiting state-of-the-art MaxSAT procedures. When applicable, these specialized procedures seem to be more efficient than general-purpose OMT. When they are not applicable, OMT-based technique can strongly benefit from the integration with bidirectional sorting networks to deal with PB components of objectives.

OMT is a young technology, with large margins for improvements. Among others, one interesting research direction is that of integrating MaxSAT-based techniques with standard OMT-based ones for efficiently handling complex objectives and constraints, so that to combine the efficiency of the former with the expressivity of the latter.

References

1. Abío, I., Nieuwenhuis, R., Oliveras, A., Rodríguez-Carbonell, E.: A parametric approach for smaller and better encodings of cardinality constraints. In: Schulte, C. (ed.) CP 2013. LNCS, vol. 8124, pp. 80–96. Springer, Heidelberg (2013). doi:10.1007/978-3-642-40627-0_9
2. Alviano, M., Dodaro, C., Ricca, F.: A maxsat algorithm using cardinality constraints of bounded size. In: Proceedings of the 24th International Conference on Artificial Intelligence, IJCAI 2015, pp. 2677–2683. AAAI Press (2015)
3. Ansótegui, C., Bofill, M., Palahí, M., Suy, J., Villaret, M., Theories, S.M.: An efficient approach for the resource-constrained project scheduling problem. In: SARA (2011)
4. Asín, R., Nieuwenhuis, R., Oliveras, A., Rodríguez-Carbonell, E.: Cardinality networks: a theoretical and empirical study. Constraints **16**(2), 195–221 (2011)
5. Barrett, C., Sebastiani, R., Seshia, S.A., Tinelli, C.: Satisfiability modulo theories, Chap. 26, vol. 185, pp. 825–885, Biere et al. [6], February 2009
6. Biere, A., Heule, M.J.H., van Maaren, H., Walsh, T. (eds.): Handbook of Satisfiability, vol. 185. IOS Press, February 2009
7. Bjorner, N.: Personal communication, 02 (2016)

8. Bjorner, N., Phan, A.-D.: νZ - maximal satisfaction with Z3. In: Proceedings of the International Symposium on Symbolic Computation in Software Science, Gammart, Tunisia, December 2014. EasyChair Proceedings in Computing (EPiC). http://www.easychair.org/publications/?page=862275542
9. Bjørner, N., Phan, A.-D., Fleckenstein, L.: νZ - an optimizing SMT solver. In: Baier, C., Tinelli, C. (eds.) TACAS 2015. LNCS, vol. 9035, pp. 194–199. Springer, Heidelberg (2015). doi:10.1007/978-3-662-46681-0_14
10. CGM-Tool. http://www.cgm-tool.eu
11. Cimatti, A., Franzén, A., Griggio, A., Sebastiani, R., Stenico, C.: Satisfiability modulo the theory of costs: foundations and applications. In: Esparza, J., Majumdar, R. (eds.) TACAS 2010. LNCS, vol. 6015, pp. 99–113. Springer, Heidelberg (2010). doi:10.1007/978-3-642-12002-2_8
12. Cimatti, A., Griggio, A., Schaafsma, B.J., Sebastiani, R.: A modular approach to MaxSAT modulo theories. In: Järvisalo, M., Van Gelder, A. (eds.) SAT 2013. LNCS, vol. 7962, pp. 150–165. Springer, Heidelberg (2013). doi:10.1007/978-3-642-39071-5_12
13. Eén, N., Sörensson, N.: Translating pseudo-boolean constraints into SAT. JSAT **2**(1–4), 1–26 (2006)
14. Larraz, D., Oliveras, A., Rodríguez-Carbonell, E., Rubio, A.: Minimal-model-guided approaches to solving polynomial constraints and extensions. In: Sinz, C., Egly, U. (eds.) SAT 2014. LNCS, vol. 8561, pp. 333–350. Springer, Heidelberg (2014). doi:10.1007/978-3-319-09284-3_25
15. Li, Y., Albarghouthi, A., Kincad, Z., Gurfinkel, A., Chechik, M.: Symbolic optimization with SMT solvers. In: POPL (2014)
16. Marques-Silva, J.P., Lynce, I., Malik, S.: Conflict-driven clause learning SAT solvers, Chap. 4, vol. 185, pp. 131–153, Biere et al. [6], February 2009
17. Nadel, A., Ryvchin, V.: Bit-vector optimization. In: Chechik, M., Raskin, J.-F. (eds.) TACAS 2016. LNCS, vol. 9636, pp. 851–867. Springer, Heidelberg (2016). doi:10.1007/978-3-662-49674-9_53
18. Narodytska, N., Bacchus, F.: Maximum satisfiability using core-guided maxsat resolution. In: Proceedings of the Twenty-Eighth AAAI Conference on Artificial Intelligence, 27–31 July, Québec City, Québec, Canada, pp. 2717–2723. AAAI Press (2014)
19. Nguyen, C.M., Sebastiani, R., Giorgini, P., Mylopoulos, J.: Multi-object reasoning with constrained goal models. Requirements Eng. 1–37 (2016). http://dx.doi.org/10.1007/s00766-016-0263-5
20. Nguyen, C.M., Sebastiani, R., Giorgini, P., Mylopoulos, J.: Requirements evolution and evolution requirements with constrained goal models. In: Comyn-Wattiau, I., Tanaka, K., Song, I.-Y., Yamamoto, S., Saeki, M. (eds.) ER 2016. LNCS, vol. 9974, pp. 544–552. Springer, Heidelberg (2016). doi:10.1007/978-3-319-46397-1_42
21. Nieuwenhuis, R., Oliveras, A.: On SAT modulo theories and optimization problems. In: Biere, A., Gomes, C.P. (eds.) SAT 2006. LNCS, vol. 4121, pp. 156–169. Springer, Heidelberg (2006). doi:10.1007/11814948_18
22. μZ. http://rise4fun.com/z3opt
23. OptiMathSAT. http://optimathsat.disi.unitn.it
24. PyLMT. http://www.bitbucket.org/stefanoteso/pylmt
25. Roussel, O., Manquinho, V.: Pseudo-boolean and cardinality constraints, Chap. 22, vol. 185, pp. 695–733 Biere et al. [6], February 2009
26. Sawaya, N.W., Grossmann, I.E.: A cutting plane method for solving linear generalized disjunctive programming problems. Comput. Chem. Eng. **29**(9), 1891–1913 (2005)

27. Sebastiani, R.: Lazy satisfiability modulo theories. J. Satisf. Boolean Model. Comput. JSAT **3**(3–4), 141–224 (2007)
28. Sebastiani, R., Tomasi, S.: Optimization in SMT with LA(Q) cost functions. In: Gramlich, B., Miller, D., Sattler, U. (eds.) IJCAR 2012. LNCS (LNAI), vol. 7364, pp. 484–498. Springer, Heidelberg (2012). doi:10.1007/978-3-642-31365-3_38
29. Sebastiani, R., Tomasi, S.: Optimization modulo theories with linear rational costs. ACM Trans. Comput. Log. **16**(2), 12 (2015)
30. Sebastiani, R., Trentin, P.: OptiMathSAT: a tool for optimization modulo theories. In: Kroening, D., Păsăreanu, C.S. (eds.) CAV 2015. LNCS, vol. 9206, pp. 447–454. Springer, Heidelberg (2015). doi:10.1007/978-3-319-21690-4_27
31. Sebastiani, R., Trentin, P.: Pushing the envelope of optimization modulo theories with linear-arithmetic cost functions. In: Baier, C., Tinelli, C. (eds.) TACAS 2015. LNCS, vol. 9035, pp. 335–349. Springer, Heidelberg (2015). doi:10.1007/978-3-662-46681-0_27
32. Sinz, C.: Towards an optimal CNF encoding of boolean cardinality constraints. In: Beek, P. (ed.) CP 2005. LNCS, vol. 3709, pp. 827–831. Springer, Heidelberg (2005). doi:10.1007/11564751_73
33. http://disi.unitn.it/trentin/resources/tacas17.tar.gz
34. Teso, S., Sebastiani, R., Passerini, A.: Structured learning modulo theories. Artif. Intell. J. **244**, 166–187 (2015). http://dx.doi.org/10.1016/j.artint.2015.04.002